Flashing a target over LoRa: operator guide

English PDF · PDF français

Français

For experimental branch `experiment/lora-ota`, checked against `tools/lora_ota/send_integrated.py`. These examples cover the TTGO LoRa32 V2.1 / SX1278 bench, not every board available in the public Web Flasher.

The full-transfer command sends the image, verifies it, activates it and confirms its boot automatically. It does not pause for operator approval between these steps.

1. Prerequisites

```
PC -- USB -- radio bridge -- LoRa -- target
                          or
PC -- USB -- radio bridge -- LoRa -- relay -- LoRa -- target
```

Device	Required role and setup
USB bridge	`bridge_fast` firmware, controller serial protocol at 921600 baud. No normal iGate Wi-Fi, TNC2 server or APRS command handling.
Target	`ENABLE_RF_OTA` firmware, two application slots and a rollback-compatible bootloader, provisioned REMOTE_AUTH key and controller callsign.
Optional relay	`ENABLE_RF_ROUTE` firmware, provisioned relay role and route. The target must also be configured for this relay.
Image	Application `firmware.bin` compatible with the board and small enough for its inactive slot. It must retain RF OTA and deferred boot confirmation.

RF transfer updates neither the bootloader nor the partition table. Initial installation requires prior access to the board. During transfer, the client opens only the bridge USB port: no target USB or Wi-Fi connection is used.

Binary source, target and relay identities are limited to **eight characters**. `--target` is the configured **Callsign-SSID**, such as `F4MLV-15`, even when the board uses the tactical APRS identity `GENATX`.

Relaying requires two separate keys: the target key for end-to-end authentication, and a transport key shared by controller, relay and target. Each is 32 bytes, represented as 43 unpadded base64url characters.

2. Prepare the workstation and files

Use a persistent checkout already on `experiment/lora-ota`. Check its state before switching branches or replacing files. Commands require Bash or Zsh, PlatformIO, Python 3 and PySerial.

```
git status --short --branch
git log -3 --oneline
python3 -c 'import serial; print(serial.__version__)'
python3 tools/lora_ota/send_integrated.py --help
```

Set paths and identities. Replace the placeholder ports. For an already provisioned chain, reuse its existing key and counter files.

```
export OTA_BRIDGE='/dev/serial/by-id/BRIDGE_PORT'
export OTA_SOURCE='F4MLV-2'
export OTA_TARGET='F4MLV-15'
export OTA_VIA='F4MLV-10'
export OTA_PRIVATE='docs/local/ota'
export OTA_KEY="$OTA_PRIVATE/target.key"
export OTA_ROUTE_KEY="$OTA_PRIVATE/route.key"
export OTA_COUNTER="$OTA_PRIVATE/F4MLV-2--F4MLV-15.counter"
export OTA_PROFILES="$OTA_PRIVATE/profiles.json"
export OTA_IMAGE='.pio/build/ttgo-lora32-v21-ota-b/firmware.bin'
export OTA_RUN="logs/radio-ota-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$OTA_PRIVATE" "$OTA_RUN"
chmod 700 "$OTA_PRIVATE" "$OTA_RUN"
```

Keep private files and logs out of publication. Retain the same counter file across runs for this controller/target pair. Use a new log path for every launch: the client opens its log in write mode.

Keys: initial provisioning only

Skip this block when keys are already installed. It refuses to overwrite an existing key file and does not print key material.

```
python3 - <<'PY'
import os, secrets, base64
from pathlib import Path
for name in ('OTA_KEY', 'OTA_ROUTE_KEY'):
    path = Path(os.environ[name])
    key = base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b'=')
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    with os.fdopen(fd, 'wb') as out:
        out.write(key)
PY
```

Radio profiles

Copy the example only if no profile file exists yet:

```
test -e "$OTA_PROFILES" || cp tools/lora_ota/radio-profiles.example.json "$OTA_PROFILES"
python3 -m json.tool "$OTA_PROFILES"
```

`normal` must match settings **already saved on the target and relay**. The repository example uses 434.100 MHz/BW125/SF12/CR4/5/2 dBm, the bench profile, not the normal 433.775 MHz network profile. `transfer` is the temporary profile: 434.550 MHz/BW250/SF8/CR4/6/3 dBm in this example.

Frequency and bandwidth are in Hz, SF is an integer, `coding_rate` is the denominator in 4/CR, and `power_dbm` is requested power. The client sends this profile over RF and configures the bridge. Do not send `OTA=FAST` manually before launching it.

3. Initial installation, only when needed

Skip this section for boards already running the prepared bench firmware. Do not reflash them between passes of the same test campaign.

Build the required variants:

```
pio run -e ttgo-lora32-v21-ota-a -e ttgo-lora32-v21-ota-b -e ttgo-lora32-v21-ota-relay
pio run -d tools/lora_ota -e bridge_fast
```

For a TTGO with 4 MiB flash, back up the entire flash before USB installation, using the actual target port:

```
export OTA_TARGET_USB='/dev/serial/by-id/TARGET_USB_PORT'
esptool.py --chip esp32 --port "$OTA_TARGET_USB" --baud 921600 read_flash 0 0x400000
        "$OTA_RUN/target-before.bin"
sha256sum "$OTA_RUN/target-before.bin" > "$OTA_RUN/target-before.sha256"
```

Also back up configurations and any other board whose firmware will be replaced. Flash sizes and offsets must be checked for other hardware.

Install the bridge and initial target firmware:

```
pio run -d tools/lora_ota -e bridge_fast -t upload --upload-port "$OTA_BRIDGE"
pio run -e ttgo-lora32-v21-ota-a -t upload --upload-port "$OTA_TARGET_USB"
```

USB installation also writes the variant's startup and partition artifacts. Check compatibility with the existing layout first. Tested boards used two 1,966,080-byte application slots.

Provision the target key and controller through firmware management. Diagnostic variants also provide serial provisioning at 115200 baud:

```
python3 -m serial.tools.miniterm "$OTA_TARGET_USB" 115200 --raw
```

Exit the monitor with `Ctrl+]` before starting a transfer. Enter the actual target key from its private file:

```
OTAPROVISION F4MLV-2 <base64url target key>
```

Expected response: `OTAPROVISIONED`. This saves the controller identity too. Do not change the key on an already provisioned target.

Optional diagnostic command `OTAOFFLINE` clears saved Wi-Fi networks, disables the AP and saves that choice. Expected response: `OTAOFFLINE_SAVED`. It is not merely a temporary Web disconnect. `OTARESET` restarts the board after these changes. Opening its USB port can reset it; do not open it during a transfer.

Adding a relay

Install the relay variant on the appropriate board once:

```
export OTA_RELAY_USB='/dev/serial/by-id/RELAY_USB_PORT'
pio run -e ttgo-lora32-v21-ota-relay -t upload --upload-port "$OTA_RELAY_USB"
```

A Web upload can also be suitable if the bootloader and partitions are already compatible. Uploading only an application cannot change their layout.

Create the route through the relay diagnostic API. Its management endpoint must actually be reachable from the workstation running this command:

```
export OTA_RELAY_URL='http://RELAY_MANAGEMENT_ADDRESS'
export OTA_RELAY_WEB_USER='MANAGEMENT_USERNAME'
python3 - <<'PY'
import os, base64
from pathlib import Path
key = Path(os.environ['OTA_ROUTE_KEY']).read_text().strip()
assert len(key) == 43 and len(base64.urlsafe_b64decode(key + '=')) == 32
path = Path(os.environ['OTA_PRIVATE']) / 'route-form.key'
path.write_text(key)
path.chmod(0o600)
PY
curl --fail --user "$OTA_RELAY_WEB_USER" \
  --data-urlencode "controller=$OTA_SOURCE" \
  --data-urlencode "target=$OTA_TARGET" \
  --data-urlencode "key@$OTA_PRIVATE/route-form.key" \
  "$OTA_RELAY_URL/lab/ota/route"
```

Curl prompts for the password. Diagnostic API credentials use Webadmin when enabled, otherwise configured OTA credentials, otherwise username `ota-lab` and the AP password. Saving the route **reboots the relay**. Without `bench_profile=1`, its saved radio configuration is retained.

Configure the target with the same transport key and relay identity:

```
OTAROUTE F4MLV-10 <base64url transport key>
OTARESET
```

Expected response before reset: `OTAROUTE_SAVED`. This target now requires its relay: removing `--via` does not turn it into a direct target. A direct target must have its route role disabled. There is no implemented serial command named `OTAROUTE=OFF`.

4. Build the image to transfer

This builds locally and does not flash any board:

```
pio run -e ttgo-lora32-v21-ota-b
test -s "$OTA_IMAGE"
sha256sum "$OTA_IMAGE" | tee "$OTA_RUN/image.sha256"
```

Send **firmware.bin**, not a merged flash image, bootloader or SPIFFS image. Keep this exact file for resumption. A valid ESP32 image header alone does not establish compatibility: the application must provide the expected RF OTA and confirmation services.

5. Run a complete transfer through one relay

Close monitors using the bridge port. Target and relay must initially listen on the JSON `normal` profile.

This command activates and confirms the received image automatically.

```
python3 tools/lora_ota/send_integrated.py \
  --bridge "$OTA_BRIDGE" \
  --source "$OTA_SOURCE" --target "$OTA_TARGET" \
  --key-file "$OTA_KEY" --counter-file "$OTA_COUNTER" \
  --via "$OTA_VIA" --route-key-file "$OTA_ROUTE_KEY" \
  --negotiate-profile --radio-profiles "$OTA_PROFILES" \
  --windowed --block-size 128 \
  --image "$OTA_IMAGE" --log "$OTA_RUN/transfer.jsonl"
```

Add `--require-offline` only if the target's Wi-Fi and AP are already disabled. The client checks this before and after updating; it does not disable them itself.

128 bytes is the default and was used for validated complete relayed transfers. New routed block sizes range from 32 to 175 bytes. Hardware trials with 175-byte blocks have so far covered 8 KiB samples, not another complete image.

6. Direct variant

For a target configured without a relay, omit `--via` and `--route-key-file`. This is the historical direct mode, acknowledging each block:

```
python3 tools/lora_ota/send_integrated.py \
  --bridge "$OTA_BRIDGE" \
  --source "$OTA_SOURCE" --target "$OTA_TARGET" \
  --key-file "$OTA_KEY" --counter-file "$OTA_COUNTER" \
  --negotiate-profile --radio-profiles "$OTA_PROFILES" \
  --image "$OTA_IMAGE" --log "$OTA_RUN/direct.jsonl"
```

It uses 204-byte blocks. Direct window mode is also available by adding `--windowed --block-size 128`; hardware trials of the direct window extension verified 8 KiB samples. `--block-size` does not change the historical mode without windows.

7. Check actual completion

The client performs authenticated HELLO, profile negotiation, transfer, whole-image verification, return to the normal radio profile, activation, reboot, new partition and SHA-256 checks, and CONFIRM. The target checks the image again for confirmation.

```
python3 - "$OTA_RUN/transfer.jsonl" <<'PY'
import json, sys
rows = [json.loads(line) for line in open(sys.argv[1])]
for row in rows:
    if row.get('event') in ('image', 'verified', 'normal_profile_confirmed',
                            'boot_pending_confirmation', 'radio_confirmed'):
        print(json.dumps(row, indent=2))
assert any(row.get('event') == 'radio_confirmed' for row in rows), 'Missing radio confirmation'
PY
```

An APRS ACK, a completed page or a byte counter reaching the image size is insufficient. Complete success includes verification and confirmation of the new boot, with a clean program exit. CONFIRM validates the RF boot and image hash, not every application sensor and service.

`block_bench.py` is a separate tool that verifies 8 or 64 KiB prefixes without activation. It does not replace complete-image validation.

8. Resume after interruption

Stop the failed sender and allow the RF inactivity lease to expire, up to 60 seconds after last activity, plus a small margin. Reuse the same keys, counter file and **exact image file**.

With `--windowed`, rerun the command with a **new log file**. WINDOW_START recovers the target's durable checkpoint: complete pages remain, while the partial page is sent again. After a relay reset, the controller refills its lost cache. Do not add `--resume-log`: it is incompatible with window mode and with profile negotiation.

A different manifest from a retained transfer causes BUSY. Resume the old image or explicitly discard it with ABORT before sending a new one.

9. Status, explicit abort and manual confirmation

The following block uses the previous variables on the normal profile. For direct operation, make `OTA_VIA` empty; otherwise retain the provisioned relay. No transfer process may be using the bridge port.

```
export OTA_ACTION='status'
python3 - <<'PY'
import os, sys, json, base64, hashlib
from pathlib import Path
sys.path.insert(0, 'tools/lora_ota')
from send_integrated import Client, STATUS, ABORT, CONFIRM, printable
from radio_profiles import load_profiles
from route_transport import RouteTransport
action = os.environ['OTA_ACTION']
assert action in ('status', 'abort', 'confirm')
key = base64.urlsafe_b64decode(Path(os.environ['OTA_KEY']).read_text().strip() + '=')
expected = hashlib.sha256(Path(os.environ['OTA_IMAGE']).read_bytes()).digest() if action ==
        'confirm' else None
c = Client(os.environ['OTA_BRIDGE'], key, os.environ['OTA_SOURCE'].encode(),
           os.environ['OTA_TARGET'].encode(), Path(os.environ['OTA_COUNTER']), lambda *a, **k:
        None)
try:
    if os.environ.get('OTA_VIA'):
        hop_key =
        base64.urlsafe_b64decode(Path(os.environ['OTA_ROUTE_KEY']).read_text().strip() + '=')
        c.route = RouteTransport(hop_key, c.source, c.target, os.environ['OTA_VIA'].encode())
    c.set_radio_profiles(load_profiles(Path(os.environ['OTA_PROFILES'])))
    c.enable_profiles()
    print(json.dumps(printable(c.hello(attempts=3)), indent=2))
    op = STATUS if action == 'status' else ABORT if action == 'abort' else CONFIRM
    reply = c.request(op, 0, expected or b'', attempts=3)
    print(json.dumps(printable(reply), indent=2))
finally:
    c.close()
PY
```

- `status` opens an authenticated session and reads status without deleting the checkpoint. The session is then left to expire.
- `abort`: set `OTA_ACTION` to `abort` and rerun the block. This explicitly discards the checkpoint and closes the session; it does not reinstall the previous firmware.
- `confirm`: set `OTA_ACTION` to `confirm`, with `OTA_IMAGE` pointing to the expected file. Use only if activation happened but the client stopped before confirmation, while the target still awaits it. The confirmation window is 180 seconds from setup start; without confirmation, firmware requests rollback. The startup watchdog can also trigger an earlier rollback.

10. APRS commands from Graywolf

Send these as authenticated commands to the correct station, through an operational APRS radio. The bench `bridge_fast` does not provide Graywolf's usual TNC2 interface.

Command to sign	Effect
`OTA=?`	OTA state, arming and prepared temporary profile
`OTA=PROFILE,434550000,250000,8,6,3`	Prepare the profile in RAM; no switch yet
`OTA=FAST,30`	Switch two seconds after the reply, with a 30-second inactivity lease
`OTA=NORMAL`	Reply on the current profile, then return to the normal profile

After FAST, the controlling radio must also use the temporary profile to reach the station. Graywolf does not configure it automatically. These commands neither carry firmware nor coordinate multiple relays: the binary client handles the transfer cycle. Do not manually switch FAST before launching it.

11. Troubleshooting and limits

Symptom	Check
No HELLO	Bridge port, key, identities of at most eight characters, normal profile, route role and board availability
BUSY at start	Pending boot confirmation or retained manifest for another image; do not discard checkpoints automatically
Rejected profile	Compatibility of every radio and burst duration below the lease; the SX1278 bridge does not support SF5/SF6
`--require-offline` fails	Target Wi-Fi/AP is still active; this option checks rather than configures it
Missing activation ACK	Check the new boot: the ACK may be lost around reset
Previous firmware returns	Check boot logs, hash and reception of CONFIRM

Do not use `--fault-checks` or `--exercise-*` for ordinary updates: they inject losses, resets or rollbacks. The transfer client does not restore board backups. Returning to the **normal radio profile** and restoring **previous firmware** are different operations.

Keep boards on the bench throughout a block-size campaign. A temporary management disconnect does not justify reflashing them. Relay HTTP access is used for diagnosis/provisioning; `send_integrated.py` does not require it to send blocks.

References: protocol and integration, hardware results, block-size comparison.